Stage 1: RTL-to-MLIR Extraction

Synthesizable Verilog Files
(Flattened Netlist)

Instruction Harness
(Instruction Names, Opcode / Funct encoding)

Modified autoGenILA

(Two-Pass: Taint Analysis & Symbolic Unrolling over Cycle Bound)

Taint-based ASV Inference

Constraint DAG Traversal (Arith, Mux, Bit-select, etc.)

New MLIR UpdateFunctionGen (Replacing LLVM iR)

ATLAAS Modification:
[1] MLIR Backend (arith, memref, scf, affine dialects)
[2] Time-indexed Input Memrefs (Grouped time series)
[3] Automation Harness (ASV Discovery, Encoding Discovery)

```
func.func @pe_update(
  %in: memref<8xi8>
) -> i32 {
  %val = load %in[%c0]
  %ext0 = extsi %val
  %ext1 = mulsi %ext0, %ext1
  %add = addsi %ext1, %acc
  return %add : i32
}
```

Per-(Instruction, ASV) MLIR functions
(func.func in arith, memref, scf, affine)

Stage 2: Semantic Lifting

atlaas-opt tool: Eight-Pass MLIR Rewrite Pipeline

Pass descriptions detailed in Table 2

[A] Bit-manip Canonicalization:

A1 canon-bitmanip (Collapses sign-extension chains to arith.extsi)
A2 narrow-types

[B] Idiom Detection:

B3 detect-mac (Walks addi to find multiplier, tags with atlaas.mac)
B4 specialize-control (Constant-folds fixed control, eliminates select chains)
B5 detect-clamp (Recognized saturation idiom ext(trunci(x)))

[C] Loop/Tensor-Shape Reconstruction:

C6 reconstruct-loops (Builds def-use chains over MAC, materialized scf.for reduction)
C7 lift-to-linalg (Lifts scf.for reduction to linalg dot-product)

[D] TAIDL Metadata Emission:

D8 emit-taidl-metadata (Annotates with taidl.* attributes, classifies arguments, infers dimensions)

```
...
%0 = arith.trunci %in : i64 to i8
%1 = arith.extsi %0 : i8 to i32
%2 = arith.addi %1, %c8 : i32
%3 = arith.ori %3, %greer : i32
%4 = arith.andi %3, %mask : i32
%5 = arith.muli %4, %w : i32
....
```

```
...
%A32 = convert(%A) : s32
%B32 = convert(%B) : s32
%dot = dot(%A32, %B32)
%mac = add(%dot, %D)
RESET %acc =
  clamp(%mac, -128, 127)
```

```
func @compute() {
  %A = load sp[@tr1]
  %B = load sp[@tr2]
  %C = linalg.matmul(%A,%B)
  %D = addi %C, %Cold
  %E = clamp %D, min, max
  store acc[@o], %E
}
```

Tensor-Level Structured MLIR
(annotated with taidl.*)

Stage 3: TAIDL Assembly

```
func @comp()
  attributes {
  op = "dot_product",
  mac_count = 256,
  clamp = true,
  grid = 16x16,
  buf = [sp, sp, acc]
}
```

Annotated MLIR
from Stage2

Python Assembler
(Materializes TAIDL Specification)

CISC Macro Composition
(canonical HLO body (matmul or conv) over config-staged state bounds)

Compute Path
(HLO templatess: mac/dot_product -> dot+add pool -> reduce(max))

DMA Path
(Driven by taidl.dma_role tags, emits load or store)

Inferring the Finite State Machine
FSM register scoring
State-update recovery
(from LLVM IR or RTL)

ACT-Consumable TAIDL Specification
(XLA-HLO Style semantics)

```
instr =
  add_inst("comp", sp,)
  instr.semantics("""
    %A = s8[16,16]
    pass(0)
    %B = s8[16,16]
    pass(1)
    %D = dot(%A, %B)
    return %D
  """)
```

```
instr =
  add_inst("comp", sp,)
  instr.semantics("""
    %A = s8[16,16]
    pass(0)
    %B = s8[16,16]
    pass(1)
    %D = dot(%A, %B)
    %E = sin(%D)
  """)
```

Hand-Written Reference

VS.

```
instr =
  add_inst("comp", sp,)
  instr.semantics("""
    %A = s8[16,16]
    pass(0)
    %B = s8[16,16]
    pass(1)
    %D = dot(%A, %B)
    return %D
  """)
```

Inferred TAIDL